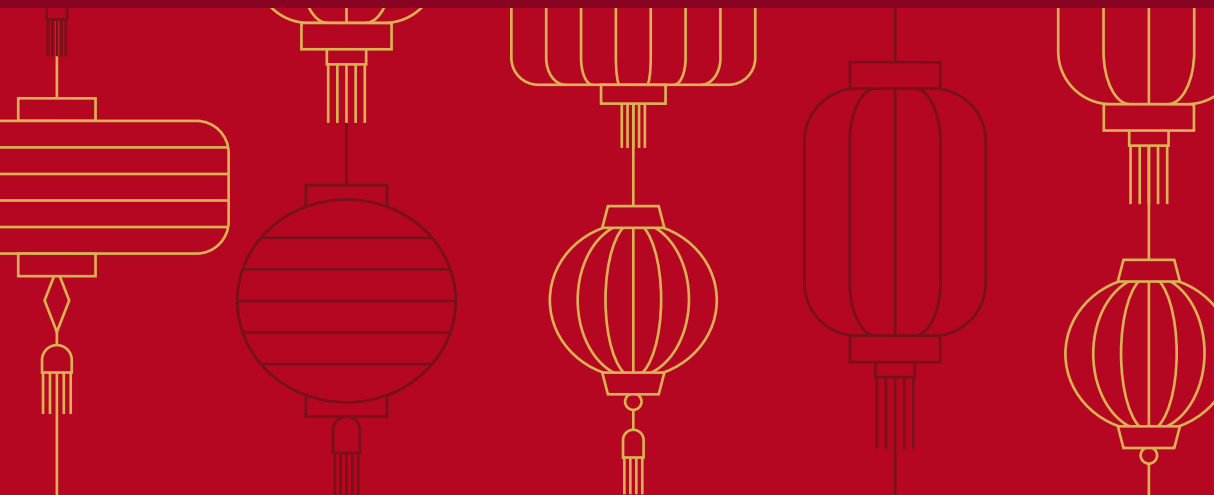




Studente(matricola, nome, cognome)

Materia(id, titolo, descrizione)

Esercizi(id, testo, soluzione, materia, numerosoluzioni)

Risolto(idesercizio, idstudente, data)

- Trovare gli studenti che non hanno risolto esercizi per "basidi dati"

$$R1 = \pi_{id-studente} \left(\sigma_{\substack{\text{titolo} = \\ \text{'basidi dati'}}} \text{Materia} \bowtie_{\text{materia} = \text{materia.id}} \text{Esercizi} \bowtie_{\text{id-esercizio} = \text{id}} \text{Risolto} \right)$$

$$\pi_{matricola}(\text{Studente}) = \pi_{\substack{id-studente \\ \rightarrow \text{matricola}}} (R1)$$

- Trovare le materie per cui sono stati risolti tutti gli esercizi

$$R1 = \pi_{id} \left[\sigma_{\substack{\text{materia} \\ \rightarrow id}} \left(\sigma_{\text{numerosoluzioni} = 0} (\text{Esercizi}) \right) \right]$$

$$\pi_{id} \text{Materia} - R1$$

SOL

✓ materia contare il numero di esercizi disponibili e quelli risolti

create view EserciziDisponibili AS

```
select materia, count(*) NumEsercizi
from Esercizi
group by materia
```

create view EserciziRisolti AS

```
select materia, count(*) NumEserRisolti
from Esercizi
group by materia
having numeroSoluzioni > 0
```

```
select materia, EserciziDisponibili.NumEsercizi, EserciziRisolti.NumEserRisolti
from EserciziDisponibili, EserciziRisolti
where Ed.materia = Er.materia
```

Trovare gli esercizi che contengono la parola 'sol' che non sono stati risolti.

```
select *  
from Esercizi  
where testo like '%sol%'  
AND numeroSoluzioni = 0
```

Trigger V ogni inserimento per un esercizio nella relazione risolto aggiornare il campo numeroSoluzioni della tabella esercizi.

```
CREATE TRIGGER AggiornaNumSoluzioni  
AFTER INSERT ON risolto  
FOR EACH ROW  
begin  
    update esercizi  
    set numeroSoluzioni = numeroSoluzioni + 1  
    where new.idesercizio = id  
end
```


Persona (cf, nome, cognome)

Libro (id, titolo, descrizione, autore, numerodilettori, datauscita, saga, volume)

Recensione (id, libro, testo, data, persona)

Letto(libro, persona, data)

• libri con almeno 2 recensioni ma che non sono stati letti

$R1 = R2 = \text{Recensione}$

$$R3 = \pi_{\substack{R1.id \\ R1.libro}} \left(R1 \bowtie_{\substack{R1.libro = R2.libro \\ R1.id > R2.id}} R2 \right)$$

libri con almeno 2
recensioni

$$R4 = \pi_{\substack{R3.id \\ R3.libro}} \left(R3 \bowtie_{R3.libro = Letto.libro} Letto \right)$$

libri con almeno 2 recensioni
che sono stati letti

$$(R3 - R4)$$

$R1 = R2 = \text{Recensione}$

$$\pi_{R1.libro} \left(R1 \bowtie_{R1.libro = R2.libro \wedge R1.id > R2.id} R2 \right) -$$

$$\pi_{libro} (Letto)$$

Trovare le persone che hanno letto tutti i libri di
J K Rowling

$$R1 = \pi_{id} \left(\sigma_{\text{Autore} = \text{'JK Rowling'}} (\text{Libro}) \right)$$

$$\pi_{id} \left(\sigma_{\substack{\text{Libro letto} \\ \text{Person} \rightarrow id}} \right) \div R1$$

SQL

vincolo che NON consenta di inserire in un Letto un
libro di una saga se non nel corretto ordine cronologico
(v1, v2, ...)

Creare Trigger T1

After
before insert ON Letto

for each row

declare x, y, z int
begin

select volume into x, saga into y

from libro

where id = New.libro

if (x is not null AND x > 1) then

select id into z

from libro

where saga = y AND volume = x - 1;

```

IF (NOT EXISTS ( select *
                  from letto
                  where Libro = z AND
                        persona = new.persona )
Then
    delete from letto where data = new.data
endif
endif
endif

```

V autore contare il num di libri e il numero di lettori
 distinti e il numero di recensioni avute da persone distinte

```

create view V1 AS
select count(*) Libri , autore
from Libro
Group By Autore

```

```

Create View V2 AS
select count (distinct persone) lettori , Autore
from Letto , Libro
where Libro = ID
Group By autore

```

```

create View V3 AS
select count (distinct persone) recensioni, autore
from Recensione , Libro
where Libro = ID
Group By Autore

```

```
select libri , recensioni , lettoni  
from v1 NATURAL JOIN v2 NATURAL JOIN v3
```

AutoPosseduta(targa, id auto, costoriifornimenti, dataimmatricolazione)

Auto(id, marca, alimentazione, cilindrata)

Rifornimento(targa, data, prezzolitro, litri)

Manutanezione(targa, data, descrizione, costo)

Algebra

- Trovare le auto, indicando marca e modello, che non sono state vendute [2 Pri]

$$R1 = \pi_{id \text{ Auto}} - \left(\sigma_{id_{auto} \rightarrow id} (AutoPossedute) \right)$$

$$\pi_{marca} (R1 \bowtie Auto) \quad \text{ou}$$

- Trovare per ogni marca le auto con la cilindrata maggiore

$$R1 = R2 = \pi_{id, marca, cilindrata} Auto$$

$$R3 = \pi_{\substack{R1.cilindrata, \\ R1.marca, \\ R1.id}} (R1 \bowtie_{\substack{R1.cilindrata \leq R2.cilindrata \\ R1.marca = R2.marca}} R2)$$

$$R4 = R1 - R3$$

(ou)

SQL

- Trovare le marche di automobili che hanno venduto tutti i modelli:

```
select distinct marca
from Auto A1
where not exists (
    select *
    from Auto A2
    where A1.marca = A2.marca
    AND NOT EXISTS ( select *
                     from AutoPossedute
                     where idauto = A2.idauto) )
```

∀ auto della marca esterna, se l'auto è posseduta, la NOT EXISTS sarà falsa.

se questo avviene ∀ auto della marca esterna la tabella^① sarà vuota e la condizione NOT EXISTS esterna sarà TRUE e la marca verrà aggiunta al risultato

- Per ogni auto posseduta mostrare quelle per le quali il numero di manutenzioni effettuate è maggiore di quello medio.
Per queste visualizzare pure il costo totale della manutenzione

create view v1 as

```
select   targa, count(*) as Num-manutenzioni, sum(costo)
        as costo
from     AutoPossedute AP, Manutenzione M
where    AP.targa = M.targa
group by targa
```

create view v2 as

```
select   avg(Num-manutenzioni) as media-man
from     v1
```

```
select   targa, Num-manutenzioni, costo
from     v1, v2
where    Num-manutenzioni > media-man
```

Forse funziona

```

Select    count(*) as num_mnt, sum(costo), Targa
From      Manutenzione

Group By  Targa

```

```

Having    num_mnt >= (
    select  avg(NumManut)
    from    (
        select count(*) as NumManut
        from    manutenzione
        group by Targa
    )
)

```

- Implementare un Trigger che ogni volta che viene inserito un rifornimento in Rifornimento aggiorna il costo complessivo in AutoPosseduta

```

create trigger T1
after insert on Rifornimento
for each row

```

```

update AutoPosseduta A

```

```

set    costoRifornimenti = costoA + (New.prezzoLitro * litri)

```

```

where  A.Targa = New.Targa

```


Libro(id, titolo, descrizione, autore, datauscita, sequeldi, genere)

CopiaLibro(collocazione, idlibro)

Persona(id, nome, cognome, prestiti)

Presitito(libro, persona, dataprestito, datarestituzione, restituito)



$$1) \quad R1 = \pi_{\text{Prestito.persona}} \left(\text{Prestito} \bowtie_{\text{Prestito.Libro = Libro.id}} \text{Libro} \right) \\ \wedge \\ \text{Libro.Autore} = \text{'Joseph Conrad'}$$

$$R2 = \pi_{\substack{\text{Prestito.} \\ \text{Persona}}} \left(\text{Prestito} \bowtie_{\text{Prestito.Libro = Libro.id}} \text{Libro} \right) \\ \wedge \\ \text{Libro.Autore} <> \text{'Stephen King'}$$

$$R1 - R2$$

$$2) \quad \rho_{\substack{\text{Libro} \\ \rightarrow \text{id}}} \left(\pi_{\text{Libro, persona}} (\text{Prestito}) \right) \quad \frac{0}{0} \quad \pi_{\text{id}} \left[\sigma_{\substack{\text{genere} = \\ \text{'Fantasy'}}} (\text{Libro}) \right]$$

ITIMEVE OS

Terreni

Terreno (id, foglio, numero, sub, dimensione)

Coltivazioni (id, nome, descrizione)

ColtivazioniTerreno (id, terreno, coltivazione, dettagli)

Raccolto (coltivazione, anno, esito, quotazione).

$$e) \quad T_1 = T_2 = \pi_{id, dimensione} (Terreno)$$

$$T_3 = \pi_{id} T_1 - \pi_{T_1.id} \left(T_1 \bowtie_{T_1.dimensione > T_2.dimensione} T_2 \right)$$

$$Q_1 = \pi_{\substack{\text{terreno,} \\ \text{quotazione}}} \left(\text{Raccolto} \bowtie_{\substack{\text{Raccolto.coltivazione} = \\ \text{coltivazioniTerreno.id} \\ \wedge \text{anno} = '2017'}} T_3 \bowtie \text{ColtivazioniTerreno} \right)$$

$$Q_2 = Q_1$$

$$Q_3 = \pi_{\substack{Q_1.terreno \\ Q_1.quotazione}} \left(Q_1 \bowtie_{Q_1.quotazione < Q_2.quotazione} Q_2 \right)$$

$$Q_4 = Q_1 - Q_3$$

b) Terreni in cui sono praticate tutte le coltivazioni indicandole tutto sul terreno

$$R1 = T_{id}(\text{coltivazioni})$$

$$R2 = \int_{\substack{\text{Coltivazione} \\ \rightarrow id}} \left(\sum_{\substack{\text{Terrens,} \\ \text{Coltivazione}}} (\text{Coltivazioni.Terreno}) \right)$$

$$\text{Terreno} \bowtie_{\text{terreno} = id} (R2 \div R1)$$

SQL

Terreni che non hanno avuto raccolti con esito positivo
per coltivazioni di colza del 2022

```
select id
from terreni T
where not exists (
```

```
select *
from coltivazioni_terreno CT, Raccolto R,
Coltivazioni C
where CT.terreno = T.id
CT.coltivazione = C.id AND
R.coltivazione = CT.id AND
R.esito = 'positivo' AND
C.nome = 'colza' AND
```

R. Anno = 2024)

b) vincolo per non inserire le stesse coltivazioni in un Terreno per più di 4 (quattro) anni.

```
create trigger T1
before insert on CollezioniTerreno
for each row
declare x NUMBER
begin
    select count(*) into x
    from CollezioniTerreno CT
    where New.Terreno = CT.Terreno AND
          New.Coltivazione = CT.Coltivazione
```

```
if (x > 4) then
```

```
    raise_application_error(20001,
```

```
    END IF
```

```
end
```

10/12/21

Persona(cf, nome, cognome, indirizzo)

Prodotto(id, tipo, descrizione, nome)

Acquisto prodotto(idprodotto, cfpersona)

TipiProdotti(id, nome)

a) Persone che non hanno effettuato acquisti dei prodotti di tipo "Fai da te"

$$R1 = \pi_{cfpersona} \left[\text{AcquistoProdotto} \bowtie_{\substack{idprodotto \\ = id}} \left(\text{Prodotto} \bowtie_{\substack{\text{TipiProdotti.id} = \text{Prodotto.tipo} \\ \wedge \\ \text{TipiProdotti.nome} = \text{"Fai da te"}}}} \right) \right]$$

$$\pi_{cfpersona} \left[\sigma_{cf \rightarrow cfpersona} (Persona) \right] - R1$$

b) Trovare i tipi dei prodotti che sono stati acquistati da tutti i clienti.

$$R1 = \pi_{\substack{cfpersona, \\ tipo}} \left(\text{Prodotto} \bowtie_{idprodotto = id} \text{AcquistoProdotto} \right)$$

$$R1 \xrightarrow{\rho} \pi_{cfpersona} (\text{AcquistoProdotto})$$

Trovare il tipo di prodotto con il massimo numero di acquisti

```
create view V1 AS
select idprodotto, count(*) conteggio
from Acquisiprodotto
group by idprodotto
```

idprodotto	conteggio
X	20
Y	30

```
select Tipo
from Prodotto, V1
where Prodotto.id = V1.idprodotto AND
V1.conteggio >= ALL
```

Assertione che non consente di inserire un acquisto di un prodotto di tipo "informatica" se non è stato acquistato già un prodotto di tipo "cucina"

create Assertion controllo

```
check (not exists (
  select *
  from
  where
  AND ~ > (select min()
```

AutoPosseduta(targa, idauto, costorifornimenti, dataimmatricolazione)

Auto(id, marca, alimentazione, cilindrata)

Rifornimento(targa, data, prezzo litro, litri)

Manutanezione(targa, data, descrizione, costo)

• Auto (marca / modello) invendute

$$\text{Auto} \quad \begin{array}{c} \diagup \quad \diagdown \\ \text{idauto} = \\ \text{id} \end{array} \left(\pi_{\text{idauto}} (\sigma_{\text{id} \rightarrow \text{idauto}}^{\text{Auto}}) - \pi_{\text{idAuto}} (\text{Auto posseduta}) \right)$$

• Trovare la marca le auto con la cilindrata maggiore

$$A_1 = A_2 = \text{Auto}$$

$$R_1 = \begin{array}{c} \pi \\ A_1: \text{id} \\ A_1: \text{marca} \end{array} \left(A_1 \begin{array}{c} \diagup \quad \diagdown \\ A_2 \end{array} \begin{array}{l} A_1: \text{marca} = A_2: \text{marca} \\ A_1: \text{cilindrata} < A_2: \text{cilindrata} \end{array} \right)$$

$$\left(\pi_{\text{id}}^{\text{marca}} (\text{Auto}) - R_1 \right)$$

SQL

Marche di auto che hanno venduto tutti i modelli

```
select distinct merce
from auto A1
where NOT EXISTS (
    select *
    from Auto A2
    where A1.merce = A2.merce
    AND NOT EXISTS (
        select *
        from Autoposseduta
        where A2.id = idauto1 )
    )
```

macchine non
vendute della
stessa marca

4 auto possedute mostrare quelle per le quali il numero di manutenzioni effettuate è maggiore di quello medio.
Per questo visualizzare pure il costo totale della manutenzione.

```
create view v1
select targa, count(*) Num_Man, sum(costo) costo_tot
from Manutenzione
group by targa
```

```
select targa, Num-Man, costo_tot
from v1
where Num-man > (select avg(Num-man)
                  from v1
                  )
```


Trigger ad ogni inserimento di rifornimento aggiorna il
costo complessivo in auto posseduta

```
create trigger T1
after insert on Rifornimento
for each row
begin
    update AutoPosseduta
    set costorifornimenti = New.Prezzolitro * New.litri
    where targa = new.targa
end
```

Negozio (id, ragionesociale, idcategoria, numero_visite)

CategorieNegozio (id, titolo, descrizione)

Città (codicelstat, nome, provincia, regione)

PresenzaNegozio (codicecitta, idnegozio, via, cap)

Persona (cf, nome, cognome)

visitaNegozio (cf, idnegozio, data)

a) Categorie Negozi presenti in tutte le provincie

$$P = \pi_{\text{provincia}} (\text{citta}^-)$$

$$R = \pi_{\text{categoria Negozi id}, (\text{citta}^- \bowtie_{\text{Codice(citta}^- = \text{codicestat})} \text{Presenza Negozio} \bowtie_{\text{id=idNegozio}} \text{Negozio} \bowtie_{\text{id=idCategoria}} \text{Categorie Negozi})}$$

citta^- provincia

$$R \div P$$

b) Le regioni in cui mancano alcune categorie di Negozi

$$C1 = \pi_{\text{id}} (\text{Categorie Negozi})$$

$$R1 = \pi_{\text{citta}^- \text{ regione}} \left(\text{Categorie Negozi} \bowtie_{\text{id categoria} = \text{id}} \text{Negozio} \bowtie_{\text{id=idNegozio}} \text{Presenza Negozio} \bowtie_{\text{Cod.istrat} = \text{cod.citta}} \text{citta}^- \right)$$

categoria Negozi id

$$R3 = R1 \div C1$$

$$\pi_{\text{regione}} (\text{citta}^-) - R3$$

sol
2) Trovare i CAP e le relative città che hanno un numero totale di negozi minore di quello medio della relativa regione

Select

Dipartimento(id, nome, direttore, numero_dipendenti, citta)

Dipendente(id, ruolo, dipartimento)

SalarioMensileDipendente(id, salario, data)

AnagraficaDipendente(id, cf, nome, cognome, data_nascita)

Algebra

a) $\forall$ dipartimento e $\forall$ ruolo trovare i dipendenti + giovani

$D1 = D2 = \text{Dipendente} \bowtie \text{AnagraficaDipendente}$

$$D3 = \pi_{D1.id,} \left(D1 \bowtie_{\begin{array}{l} D1.dip = D2.dip \\ \wedge \\ D1.ruolo = D2.ruolo \\ \wedge \\ D1.data_nascita < D2.data_nascita \end{array}} D2 \right)$$

$$\pi_{id} (Dipendente) - D3$$

sal

a) $\forall$ dipartimento trovare il numero di dipenso